Progress Report 3

Intelligent Distributed Compute Fabric for Heterogeneous Personal Devices

25 September – 7 October 2026

Rishu Raj (23UCS085) · Abhay Singh (23UCS097) · Aayush Kumar (23UCS095) · Aditya Kumar Tiwari (23UCS099)

1 SUMMARY

In these two weeks the system went from "laptops can connect" to **"a task can be sent to another laptop, run safely, and survive failures"**. The first way of running work — sending a complete task to another laptop — is now built end to end on the software side.

- About **7,200 automatic tests** now run on the system (up from 2,600).
- **370+ code commits** in this period (616 in total).
- Every important design choice is still written down so it can be explained later.

2 HOW THE SYSTEM WORKS

There are three parts:

- **Main laptop (controller)** — the "manager". It knows every laptop, keeps all records in a database, and decides where each task runs.
- **Background program (agent)** on every other laptop — the "worker". It reports its hardware and how busy it is, runs tasks, and sends results back.
- **Command line** (`compute run`, `compute logs`, `compute get`) — what the user types. A web dashboard comes later.

Flow: user submits a task → the manager picks the best laptop → that laptop gets the files → runs the task in a sealed box → sends the result back.

3 WHAT WE USED

- **Python** — main language; AI workloads are already in Python.
- **gRPC + Protocol Buffers** — fast, ready-made way for laptops to talk, with a fixed message format.
- **FastAPI** — web interface used by the command line (and later the dashboard).
- **PostgreSQL** — database for all records, so nothing is lost if the main laptop crashes.
- **mDNS** — laptops find the main laptop automatically, like a Wi-Fi printer.
- **psutil, NVML** — read CPU, memory, disk and NVIDIA graphics card details.
- **Docker** (or **uv** Python setups) — run the task in a sealed box so it cannot harm the laptop.
- **SHA-256 fingerprints + HTTP** — send only missing files and detect damaged ones.
- **Public/private keys (ECDSA, Ed25519)** — each laptop proves who it is.
- **Typer** — the `compute` commands; **pytest** — automatic tests.

4 WHAT WE BUILT IN THIS PERIOD

Sending a task to another laptop

- A user runs `compute run`; the task is sent to another laptop, runs there, and the result comes back with `compute get`.
- The output (logs) can be watched live while the task runs.
- The task runs inside a sealed Docker box with limited memory and no network. Where Docker is not available, a separate Python setup is used.
- A task can be cancelled at any time, and every failure is reported with a clear reason (out of memory, disk full, time limit, etc.).

Moving code and data efficiently

- Every file is identified by a fingerprint of its contents, so a file is never sent twice.
- Laptops fetch files directly from each other, not only from the main laptop. If one source is slow or wrong, another is used.
- Large files resume from where they stopped if the network drops. Every piece is checked, so a damaged file is never used.
- Python setups are built once per machine and then reused.

Choosing the machine

- The planner lists the laptops that can run the task, removes those that cannot, and picks the best one.
- `compute explain` shows why a machine was chosen or rejected.
- Laptops now measure the network speed between each other. The planner uses this to estimate how long a task would take on each machine, including the time to send its data. For example, a small task with a lot of data stays on the user's own laptop if the network is slow.

Surviving failures

- If a laptop is switched off or loses Wi-Fi, its task is automatically restarted on another laptop.
- If the main laptop crashes, it recovers on restart and no task is run twice. Tested by force-killing it with 20 tasks running.
- If a disconnected laptop comes back, its old results are safely ignored.
- A **"chaos test"** randomly kills laptops, cuts the network and fills disks while 50 tasks run, and checks that nothing is lost or duplicated. It passes.

5 HOW WE CHECK IT WORKS

Each part still goes through three steps: one writes the code, a second tries to break it with its own tests, and a third reviews and must approve. In this period the testing step found real bugs — for example, a laptop with a full disk could make many tasks fail; now the task simply moves to another laptop.

All of this is tested on one machine with several simulated laptops. Real tests on our four laptops are the next step.

6 WHERE WE ARE IN THE PLAN

- **Done:** base setup, core connection between laptops, sending a task away, moving code and data.
- **Almost done:** choosing machines and handling failures — the last part (network measurement and time estimates) is being finished and tested now.
- **Next:** security (only approved laptops can join, encrypted connections), then the dashboard, then splitting a task across many laptops.

7 WHAT WE DO NEXT, AND HOW

1. **Real test on 4 laptops** — run 100 tasks while switching off laptops, turning off Wi-Fi and crashing the main laptop; check nothing is lost or repeated.
2. **Security** — only approved laptops can join (one-time join code); all connections encrypted with **mTLS** (like HTTPS, but both sides show an ID card).
3. **Dashboard** — a **React** web page showing laptops, tasks and logs.
4. **Measurements** — record how long every task took on every laptop; this becomes the data for the machine learning.
5. **Splitting one big task** — cut the work into small pieces in one shared list; each free laptop takes a piece, a faster laptop simply takes more. If a laptop disappears, its piece goes back to the list.
6. **Smart planner** — a model (**LightGBM / scikit-learn**) trained on our own past runs predicts task time, and the planner picks the fastest option.
7. **Using other laptops' graphics cards** — already designed, added later. Done inside **PyTorch**: the numbers go to the other laptop's GPU and only the answer comes back.

8 WHY THIS IS MORE THAN SSH OR DOCKER

SSH only runs a command on another computer. Our system **chooses the laptop itself**, **moves files smartly**, **recovers when a laptop dies**, and will **learn which laptop is fastest** for each kind of task.

9 WHAT WE NEED

- **Time with all four laptops together** for the real-machine tests.
- **A network switch and cables** for reliable speed measurements.